第 3 章推導解答

DS03-01 • 環狀回繞

初始 $(head, size) = (0, 0)$。表中「—」表示沒有有效元素；若底層仍殘留舊字值，也不屬於抽象序列。enqueue 的槽為 $(head + size) \bmod 4$，dequeue 後 $head = (head + 1) \bmod 4$。

已完成操作	head	size	槽0	槽1	槽2	槽3	邏輯序列
初始	0	0	—	—	—	—	空
enqueue A	0	1	A	—	—	—	A
enqueue B	0	2	A	B	—	—	A,B
enqueue C	0	3	A	B	C	—	A,B,C
dequeue A	1	2	—	B	C	—	B,C
dequeue B	2	1	—	—	C	—	C
enqueue D	2	2	—	—	C	D	C,D
enqueue E	2	3	E	—	C	D	C,D,E
enqueue F	2	4	E	F	C	D	C,D,E,F
dequeue C	3	3	E	F	—	D	D,E,F
dequeue D	0	2	E	F	—	—	E,F
dequeue E	1	1	—	F	—	—	F

Lin Cing https://github.com/xinhi6716/computer-science-self-study

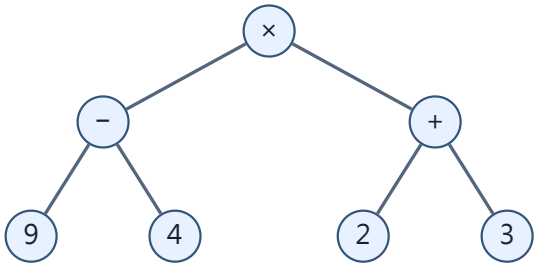
滿時 size=4；空時 size=0。這個表示法不靠 head 與某個 tail 索引相等來區分空滿。

DS03-02 • 運算式轉換

樹根為 *，左子樹根 -、葉 9 與 4，右子樹根 +、葉 2 與 3。

第 3 章推導解答 （續）

(9−4)×(2+3)：左右子樹決定運算元順序



前序：* − 9 4 + 2 3

後序：9 4 − 2 3 + *

左子式 9−4=5；右子式 2+3=5

最後 5×5=25

二元運算：先 pop 右值，再 pop 左值

前序為 `* - 9 4 + 2 3`，後序為 `9 4 - 2 3 + *`。後序堆疊依序為 `[9]`、`[9,4]`、`[5]`、`[5,2]`、`[5,2,3]`、`[5,5]`、`[25]`，即先算 9−4，再算 2+3，最後相乘。`8 3 -` 首先 pop 的 3 是右運算元，第二次的 8 是左運算元，因此得 5。交換會得 −5，違反原運算樹語義。

DS03-03 • DFS 的第一條路徑

取有向邊 `s→a,a→b,b→c,c→t,s→t`，s 的鄰居依 a、t 排序。DFS 依序訪問 s、a、b、c、t，第一次抵達路徑長 4，parent 為 a←s、b←a、c←b、t←c。BFS 在處理 s 時已發現 a 與 t，t 的 parent=s、距離 1。若完整繼續，佇列訪問可為 s、a、t、b、c。反例只依賴 DFS 的鄰居次序，不否定 DFS 可達性正確性。

此 BFS 若完整執行，parent 為 a←s、t←s、b←a、c←b；距離依序 d(s)=0、d(a)=d(t)=1、d(b)=2、d(c)=3。父指標在首次發現時設定，後續遇到已發現的 t 不改成 c。

DS03-04 • 語法與求值安全

`2 +` 在讀到二元 + 時堆疊不足兩值，立即拒絕。`2 3` 掃描完剩兩值，違反單一結果條件。`8 0 /` 取得右運算元 0 時拒絕零除。另需檢查整數剖析範圍、加減乘溢位，以及最小負整數除以 −1 的不可表示結果。若語法含一元負號，其 arity 必須由 token 類型確定，不能以當時堆疊大小猜測。

DS03-05 • 掃描收尾與文法

堆疊／輸出依序為：9→`∅ / 9`；−→`- / 9`；4→`- / 9 4`；*→`- * / 9 4`；2→`- * / 9 4 2`；/→`- / / 9 4 2 *`；1→`- / / 9 4 2 * 1`；+→`+ / 9 4 2 * 1 / -`；3→`+ / 9 4 2 * 1 / - 3`；EOF→`∅ / 9 4 2 * 1 / - 3 +`。在 + 時須先彈 / 再彈 −，結果為4。`(9-4` 在EOF清空後仍有 (而拒絕；`9-4)` 在) 時找不到 (而拒絕；`9-` 在EOF仍期待運算元而拒絕。右結合後序 `2 3 2 ^ ^` 得512；左結合後序 `2 3 ^ 2 ^` 得64。

第 3 章推導解答 （續）

DS03-06 • 佇列擴容的抽象值

C 由舊槽2到新槽0，D 由3到1，E 由0到2，F 由1到3；提交後 head=0,size=4,capacity=8。直接複製舊四格並留head=2，邏輯第2項會讀新槽 `(2+2) mod 8=4`，它未建構而不是E，故抽象序列已錯。按邏輯順序搬動一次為Θ(n)，某次入列最壞Θ(n)；僅在滿時倍增且元素搬移有常數上界，總搬移形成幾何級數，從空的n次入列總Θ(n)、攤銷Θ(1)。若值移動成本不固定則需另乘該成本。固定容量最壞Θ(1)保證不能與這個攤銷保證混寫。

原創驗算程式附錄: expression_reference.py

```
from fractions import Fraction
import json
import random
import re

PRECEDENCE = {'+':1, '-':1, '*':2, '/':2, '^':3}

def tokenize(expression):
    tokens = re.findall(r'\d+|[()+*/^--]', expression)
    if re.sub(r'\s+', '', expression).replace('−', '-') != ''.join(tokens).replace('−', '-'):
        raise ValueError('unknown token')
    return [token.replace('−', '-') for token in tokens]

def postfix(expression):
    output, stack, trace = [], [], []
    operand_expected = True
    for token in tokenize(expression):
        if token.isdigit():
            if not operand_expected:
                raise ValueError('missing operator')
            output.append(token)
            operand_expected = False
        elif token == '(':
            if not operand_expected:
                raise ValueError('missing operator before parenthesis')
            stack.append(token)
        elif token == ')':
            if operand_expected:
                raise ValueError('missing operand')
            while stack and stack[-1] != '(':
                output.append(stack.pop())
            if not stack:
                raise ValueError('unmatched right parenthesis')
```

Lin Cing https://github.com/xinhi6716/computer-science-self-study

第 3 章推導解答 （續）

```
            stack.pop()
            operand_expected = False
        else:
            if operand_expected:
                raise ValueError('binary operator needs left operand')
            while stack and stack[-1] != '(' and (
                PRECEDENCE[stack[-1]] > PRECEDENCE[token] or
                (PRECEDENCE[stack[-1]] == PRECEDENCE[token] and token != '^')):
                output.append(stack.pop())
            stack.append(token)
            operand_expected = True
        trace.append((token, ' '.join(stack) or 'Ø', ' '.join(output) or 'Ø'))
    if operand_expected:
        raise ValueError('empty expression or missing final operand')
    while stack:
        if stack[-1] == '(':
            raise ValueError('unmatched left parenthesis')
        output.append(stack.pop())
    trace.append(('EOF', 'Ø', ' '.join(output)))
    return output, trace

def apply(op, a, b):
    if op == '+': return a+b
    if op == '-': return a-b
    if op == '*': return a*b
    if op == '/': return a/b
    if b.denominator != 1 or abs(b) > 20:
        raise ValueError('exponent outside demonstration domain')
    return a**int(b)

def evaluate_postfix(tokens):
    stack = []
```

Lin Cing https://github.com/xinhi6716/computer-science-self-study

第 3 章推導解答 （續）

```
    for token in tokens:
        if token.isdigit():
            stack.append(Fraction(token))
        else:
            if len(stack) < 2: raise ValueError('operand underflow')
            b, a = stack.pop(), stack.pop()
            stack.append(apply(token,a,b))
    if len(stack) != 1: raise ValueError('extra operands')
    return stack[0]

def parse_reference(expression):
    tokens = tokenize(expression)
    position = 0
    def parse(minimum=0):
        nonlocal position
        token = tokens[position]
        position += 1
        if token == '(':
            value = parse()
            if tokens[position] != ')': raise ValueError('reference parenthesis')
            position += 1
        elif token.isdigit():
            value = Fraction(token)
        else:
            raise ValueError('reference operand')
        while position < len(tokens) and tokens[position] in PRECEDENCE and
PRECEDENCE[tokens[position]] >= minimum:
            op = tokens[position]
            position += 1
            rhs = parse(PRECEDENCE[op] + (op != '^'))
            value = apply(op, value, rhs)
        return value
    result = parse()
```

Lin Cing https://github.com/xinhi6716/computer-science-self-study

第 3 章推導解答 （續）

```
    if position != len(tokens): raise ValueError('reference leftover')
    return result

def main():
    cases = ['9-4', '(9-4)*(2+3)', '9-4*2/1+3', '2^3^2', '9-4-2']
    assert postfix('9-4')[0] == ['9','4','-']
    assert postfix('9-4*2/1+3')[0] == ['9','4','2','*','1','/','-','3','+']
    assert postfix('2^3^2')[0] == ['2','3','2','^','^']
    legacy_output_before_eof = postfix('9-4')[1][-2][2]
    assert legacy_output_before_eof == '9 4'
    invalid = ['(9-4','9-4)','9-','9 4','()','9+(4*)','9**4', '', '(9)(4)']
    for expression in invalid:
        try: postfix(expression)
        except ValueError: pass
        else: raise AssertionError(expression)
    rng = random.Random(20261024)
    def expression(depth):
        if depth == 0 or rng.randrange(3) == 0: return str(rng.randrange(1,10))
        return '('+expression(depth-1)+rng.choice('+-*')+expression(depth-1)+')'
    generated = [expression(5) for _ in range(500)]
    for value in cases+generated:
        assert evaluate_postfix(postfix(value)[0]) == parse_reference(value)
    print(json.dumps({'legacy_omitted_EOF_output':legacy_output_before_eof,
                      'corrected_postfix_9_minus_4':' '.join(postfix('9-4')[0]),
                      'valid_reference_comparisons':len(cases+generated),
                      'invalid_inputs_rejected':len(invalid),
                      'trace':{case:postfix(case)[1] for case in
cases}},ensure_ascii=False,indent=2))

if __name__ == '__main__':
    main()
```

Lin Cing https://github.com/xinhi6716/computer-science-self-study

參考資料與署名

Ellis Horowitz, Sartaj Sahni, Dinesh Mehta. Fundamentals of Data Structures in C++, 2nd Edition.

Lin Cing 原創教學整理。原創內容以 CC BY 4.0 授權；參考書籍不隨教材散布。
https://github.com/xinhi6716/computer-science-self-study